

<https://github.com/GayeonLee03/2024125074.git>

# SafeRoute

## 소프트웨어 설계서

---

군인·피난민을 위한 전쟁 시 위험지역 신고 지도 및  
실시간 안전 경로 안내 서비스

| 항목                | 내용                                                                                                        |
|-------------------|-----------------------------------------------------------------------------------------------------------|
| 소속                | 한국항공대학교 소프트웨어학과                                                                                           |
| 팀명                | SafeRoute                                                                                                 |
| 작성자               | 이가연                                                                                                       |
| 작성일               | 2026.05.25                                                                                                |
| 문서 버전             | 1.1                                                                                                       |
| GitHub Repository | <a href="https://github.com/GayeonLee03/2024125074.git">https://github.com/GayeonLee03/2024125074.git</a> |

Copyright 2026. SafeRoute. All rights reserved.

## 제/개정 이력

| 버전  | 날짜         | 작성자 성명 | 제/개정사항                                 | 비고    |
|-----|------------|--------|----------------------------------------|-------|
| 1.0 | 2026.05.25 | 이가연    | SafeRoute 소프트웨어 설계서 최초 작성              | 신규 작성 |
| 1.1 | 2026.05.25 | 이가연    | 표 중심 구성을 줄이고 서술형 설명을<br>보강하여 문서 가독성 개선 | 개정    |

## 문서 관리 정보

본 문서는 SafeRoute 프로젝트의 구현 단계에서 기준으로 활용하기 위한 소프트웨어 설계서이다. 문서의 주요 독자는 프로젝트 개발자, 기획 담당자, 테스트 담당자, 검토자 및 지도 교수이며, 설계 변경이 발생할 경우 문서 버전과 제/개정 이력을 함께 갱신한다.

문서에는 소프트웨어 아키텍처, 모듈/패키지 설계, 인터페이스 설계, 데이터 설계, 구현 기술 설계, 요구사항 추적표가 포함된다. 또한 기존에 작성한 시스템 계획서, 요구사항 정의서, 요구사항 분석서의 내용을 연결하여 같은 프로젝트 흐름을 유지하도록 작성하였다.

### 문서 작성 방향

- 샘플 소프트웨어 설계서의 기본 목차와 설계 항목을 참고한다.
- 기존 SafeRoute 과제 문서의 문체처럼 설명 문단을 중심으로 작성하고, 필요한 부분만 표로 정리한다.
- 무채색 계열의 표 머리글과 간결한 여백을 사용하여 과제 제출용 문서의 가독성을 높인다.

# 목차

---

## 1. 서론

- 1.1 목적 및 범위
- 1.2 용어 정의
- 1.3 참조 문서

## 2. 소프트웨어 아키텍처

- 2.1 정적 구조
- 2.2 동적 구조

## 3. 모듈/패키지 설계

- 3.1 Client/UI 패키지
- 3.2 User/Auth 패키지
- 3.3 Risk Report 패키지
- 3.4 Map/Route 패키지
- 3.5 Shelter/Notification 패키지
- 3.6 Admin/Data 패키지

## 4. 인터페이스 설계

- 4.1 외부 시스템 인터페이스
- 4.2 사용자 인터페이스

## 5. 데이터 설계

## 6. 구현 기술 설계

## 7. 요구사항 추적표

## 8. 부록

# 1. 서론

## 1.1 목적 및 범위

본 문서는 SafeRoute 시스템의 소프트웨어 설계 내용을 정의하기 위한 문서이다. SafeRoute는 전쟁 및 재난 상황에서 군인, 피난민, 구조기관이 위험지역 정보를 신속하게 공유하고 위험지역을 회피한 안전 경로를 안내받을 수 있도록 지원하는 지도 기반 서비스이다.

기존 시스템 계획서에서는 프로젝트의 개발 배경, 필요성, 주요 기능, 개발 환경을 정의하였고, 요구사항 정의서에서는 기능적·비기능적·인터페이스 요구사항을 정리하였다. 또한 요구사항 분석서에서는 Use Case, 정적 분석, CRC 카드, 동적 분석, 인터페이스 분석을 통해 시스템을 분석하였다. 본 설계서는 이러한 이전 산출물을 바탕으로 실제 구현에 필요한 패키지 구조, 클래스 책임, 데이터 저장 방식, 외부 연동 방식, UI 흐름, 예외 처리 방향을 구체화한다.

문서의 범위는 과제 요구사항에 따라 서론, 소프트웨어 아키텍처, 모듈/패키지 설계, 인터페이스 설계, 데이터 설계, 구현 기술 설계, 요구사항 추적표를 포함한다. 각 장은 개발자가 구현 단계에서 직접 참고할 수 있도록 설계 의도와 처리 흐름을 설명 중심으로 작성하였으며, 클래스 목록이나 요구사항 추적 관계처럼 비교가 필요한 항목은 표로 정리하였다.

## 1.2 용어 정의

| 용어         | 설명                                           |
|------------|----------------------------------------------|
| SafeRoute  | 위험지역 신고와 안전 경로 안내를 제공하는 본 프로젝트의 시스템 명칭       |
| 위험지역       | 폭격, 교전, 지뢰, 붕괴, 화재 등 사용자의 안전을 위협할 수 있는 지역    |
| 안전경로       | 위험지역을 최대한 회피하여 목적지까지 이동할 수 있도록 추천되는 경로       |
| 위험도        | 위험지역의 심각도를 낮음, 중간, 높음 등급으로 표시한 값             |
| GPS        | 위성 기반 위치 확인 기술로 사용자의 현재 위치를 파악하는 데 활용되는 기술   |
| 지도 API     | 지도 표시, 위치 검색, 경로 탐색, 마커 표시 기능을 제공하는 외부 인터페이스 |
| 신고 데이터     | 사용자가 입력한 위험 유형, 위치, 사진, 설명, 신고 시간 등의 정보      |
| 대피소        | 위험 상황에서 사용자가 임시로 대피하거나 보호받을 수 있는 장소          |
| Use Case   | 사용자 또는 외부 시스템이 SafeRoute와 상호작용하여 수행하는 기능 단위  |
| Repository | 데이터베이스 접근 기능을 분리하여 저장, 조회, 수정, 삭제를 담당하는 객체   |

## 1.3 참조 문서

본 설계서 작성에는 이전 과제로 작성한 SafeRoute 시스템 계획서, 요구사항 정의서, 요구사항 분석서를 주요 기준 문서로 활용하였다. 시스템 계획서는 프로젝트의 전체 방향과 개발 배경을 확인하는 데 사용하였고, 요구사항 정의서는 기능 요구사항과 검수 기준을 설계 요소로 연결하는 데 활용하였다. 요구사항 분석서는 Use Case, 소프트웨어 문맥도, CRC 카드, 동적 분석을 참고하여 패키지와 클래스의 책임을 정리하는 데 사용하였다.

또한 과제에서 제공된 Sample-과제 5 소프트웨어 설계서는 목차 구성, 모듈/패키지 설계 항목, 인터페이스 설계 항목, 데이터 설계 항목의 기준으로 참고하였다. 지도 API, GPS 위치 서비스, GitHub Repository 관리 자료는 외부 시스템 연동과 산출물 관리 방식을 정리하는 보조 자료로 활용하였다.

## 2. 소프트웨어 아키텍처

SafeRoute 는 사용자 단말기, 클라이언트 UI, 백엔드 서버, 데이터베이스, 외부 지도/GPS API, 대피소 데이터, 알림 서비스가 상호작용하는 구조로 설계한다. 사용자는 웹 또는 모바일 화면을 통해 위험지역 신고, 현재 위치 확인, 안전 경로 탐색, 대피소 조회, 신고 내역 확인 기능을 수행한다. 서버는 사용자의 요청을 검증하고, 데이터베이스와 외부 API 를 연동하여 필요한 결과를 생성한 뒤 다시 클라이언트 화면에 전달한다.

아키텍처 설계의 핵심은 기능별 책임을 분리하는 것이다. Client/UI 패키지는 화면과 입력을 담당하고, User/Auth 패키지는 인증과 권한을 담당한다. Risk Report 패키지는 위험지역 신고와 위험 구역 갱신을 처리하며, Map/Route 패키지는 현재 위치 확인, 지도 시각화, 안전 경로 탐색을 담당한다. Shelter/Notification 패키지는 대피소 정보와 알림을 처리하고, Admin/Data 패키지는 신고 검증, 사용자 관리, 로그와 백업을 담당한다.

### 2.1 정적 구조

정적 구조는 시스템이 어떤 구성 요소로 이루어지는지와 각 구성 요소가 어떤 책임을 가지는지를 나타낸다. SafeRoute 는 크게 표현 계층, 응용 계층, 연동 계층, 데이터 계층으로 나눌 수 있다. 표현 계층은 사용자와 직접 맞닿아 있는 화면을 담당하며, 응용 계층은 실제 업무 규칙과 기능 처리를 담당한다. 연동 계층은 지도 API, GPS 서비스, 공공 데이터, 이미지 저장소, 푸시 알림 서비스와 연결되고, 데이터 계층은 사용자 정보와 위험지역 신고, 경로 이력, 대피소 정보, 로그를 저장한다.

#### 정적 구조 개요

- 사용자 단말기 → Client/UI → Backend API → Service Layer → Repository → Database
- Service Layer 는 필요에 따라 External API Adapter 를 통해 지도 API, GPS, 대피소 데이터, 알림 서비스와 통신한다.
- 모든 주요 데이터는 Repository 를 거쳐 저장되며, 관리자 조치와 오류 정보는 AuditLog 에 기록된다.

| 패키지                  | 주요 책임                          | 관련 기능                    |
|----------------------|--------------------------------|--------------------------|
| Client/UI            | 화면 표시, 사용자 입력 수집, 지도와 경로 출력    | 로그인, 신고, 지도, 경로, 관리자 화면  |
| User/Auth            | 회원가입, 로그인, 사용자 유형 관리, 권한 검증    | 사용자 및 권한 관리              |
| Risk Report          | 위험지역 신고 등록, 신고 중복 판단, 위험 구역 갱신 | 위험지역 신고, 위험지역 조회         |
| Map/Route            | 현재 위치 확인, 지도 표시, 안전 경로 계산, 재탐색 | GPS 위치 확인, 지도 시각화, 경로 안내 |
| Shelter/Notification | 대피소 조회, 안전 지역 안내, 위험 접근 알림     | 대피소 정보, 경로 재탐색 알림        |
| Admin/Data           | 신고 검증, 사용자 제한, 데이터 백업, 로그 관리   | 관리자 관리, 데이터 관리           |

정적 구조에서 가장 중요한 설계 기준은 위험지역 신고 데이터와 지도 표시 데이터를 분리하는 것이다. 사용자가 신고한 개별 데이터는 RiskReport 로 저장되지만, 지도에는 같은 지역의 유사 신고를 하나로 묶은 DangerZone 이 표시된다. 이러한 구조를 사용하면 동일 위치의 반복 신고를 지도에 과도하게 표시하지 않고, 위험도의 변화와 신고 수를 반영하여 더 보기 좋은 화면을 제공할 수 있다.

또한 사용자 유형별 권한을 분리하기 위해 User/Auth 패키지와 AccessPolicy 클래스를 둔다. 피난민과 군인은 신고와 경로 탐색을 중심으로 기능을 사용하고, 구조기관은 위험지역과 대피소 정보를 확인하는 기능을 주로 사용한다. 관리자는 신고 검증, 사용자 제한, 로그 확인 등 운영 기능에 접근할 수 있다.

## 2.2 동적 구조

동적 구조는 사용자가 기능을 실행할 때 시스템 내부 구성 요소가 어떤 순서로 동작하는지를 설명한다. SafeRoute 의 핵심 동작은 위험지역 신고, 현재 위치 및 주변 위험지역 확인, 안전 경로 탐색, 이동 중 경로 재탐색으로 구분된다.

### 2.2.1 위험지역 신고 흐름

사용자가 위험지역 신고 버튼을 선택하면 Client/UI 는 현재 위치를 자동으로 가져오거나 사용자가 지도에서 선택한 위치를 신고 위치로 설정한다. 사용자는 위험 유형, 위험도, 사진, 설명을 입력하고 신고하기 버튼을 누른다. 이때 클라이언트는 필수 입력값을 1 차로 확인하고, 서버의 ReportService 는 좌표, 위험 유형, 위험도, 첨부 파일 형식 등을 다시 검증한다.

검증이 완료되면 RiskRepository 는 신고 데이터를 데이터베이스에 저장하고 신고 접수 시간을 자동으로 기록한다. 이후 DangerZoneService 는 같은 위치에 유사 신고가 있는지 확인한다. 유사 신고가 존재하면 기존 위험 구역에 병합하고, 존재하지 않으면 새로운 위험 구역을 생성한다. 마지막으로 MapService 는 갱신된 위험 구역 정보를 지도 화면에 반영하고, NotificationService 는 해당 위험지역 근처 사용자에게 경고 알림을 보낼 수 있다.

### 2.2.2 안전 경로 탐색 흐름

사용자가 출발지와 목적지를 입력하면 RouteService 는 먼저 입력값이 올바른 좌표인지 확인한다. 이후 현재 활성화된 위험지역 목록을 조회하고, 외부 지도 API 를 통해 후보 경로를 받아온다. SafeRouteEngine 은 각 후보 경로가 위험지역과 얼마나 가까운지, 위험도가 높은 지역을 통과하는지, 대피소를 경유할 수 있는지 등을 기준으로 위험 점수를 계산한다.

계산 결과 위험 점수가 낮은 경로가 우선 추천된다. 사용자 화면에는 추천 경로가 지도 위 선으로 표시되고, 예상 거리와 예상 이동 시간이 함께 제공된다. 만약 완전히 안전한 경로가 존재하지 않는 경우에는 가장 위험도가 낮은 대체 경로를 안내하고, 위험 요소가 포함되어 있다는 경고를 함께 표시한다.

### 2.2.3 이동 중 재탐색 및 알림 흐름

사용자가 경로 안내를 실행 중일 때 LocationService 는 사용자의 현재 위치를 주기적으로 갱신한다. 동시에 RouteService 는 현재 이동 경로와 최신 위험지역 데이터를 비교한다. 새롭게 등록된 위험지역이 현재 경로와 겹치거나 사용자가 일정 거리 이내로 접근하면 NotificationService 가 경고 알림을 생성한다.

사용자는 알림을 확인한 뒤 경로 재탐색을 선택할 수 있다. 위험도가 높은 긴급 상황에서는 시스템이 자동으로 대체 경로를 제안할 수 있다. 이 과정에서 기존 경로, 새로운 위험지역, 대피소 위치가 함께 고려되며, 재탐색 결과는 지도 화면에 즉시 반영된다.

### 2.2.4 신고 상태 전이

위험지역 신고는 접수, 검토 중, 승인, 중복 병합, 보류, 삭제 또는 해제 상태로 전이된다. 사용자가 신고를 등록하면 기본 상태는 접수로 저장된다. 동일 위치에 유사한 신고가 존재하면 중복 병합 상태로 처리되어 기존 DangerZone 의 신고 수와 위험도가 갱신된다. 관리자가 신고 내용을 검토한 뒤 신뢰할 수 있는 신고라고 판단하면 승인 상태가 되고, 허위 가능성이 있거나 정보가 부족하면 보류 상태가 된다. 위험이 사라지거나 허위 신고로 판단되면 삭제 또는 해제 상태로 변경되며, 모든 조치 내역은 감사 로그에 기록된다.

### 3. 모듈/패키지 설계

본 장에서는 SafeRoute 를 구성하는 주요 패키지별 상세 설계를 정의한다. 기존 문서처럼 모든 내용을 표로 나열하기보다는 각 패키지의 역할과 설계 의도를 먼저 문장으로 설명하고, 클래스 목록과 핵심 메소드만 표로 정리한다. 이를 통해 개발자가 패키지의 목적을 이해한 뒤 상세 항목을 확인할 수 있도록 구성하였다.

#### 3.1 SDD-P-001 : Client/UI 패키지

Client/UI 패키지는 사용자와 시스템이 직접 만나는 표현 계층이다. 로그인 화면, 메인 지도 화면, 위험지역 신고 화면, 안전 경로 탐색 화면, 대피소 정보 화면, 관리자 화면을 제공한다. 이 패키지는 복잡한 비즈니스 로직을 직접 처리하지 않고, 사용자의 입력값을 수집하여 서버에 전달하고 서버 응답을 화면에 보기 좋게 표시하는 역할에 집중한다.

##### 주요 책임

- 화면 전환과 사용자 입력 이벤트 처리
- 지도 위 마커, 위험 구역, 추천 경로 출력
- 신고 입력값의 1차 검증
- 서버 오류 또는 권한 오류 메시지 출력

##### 구성 클래스

| 클래스          | 설명               | 주요 메소드                                         |
|--------------|------------------|------------------------------------------------|
| UIController | 화면 전환과 공통 이벤트 제어 | navigate(), showError(), showLoading()         |
| MapView      | 지도, 위험지역, 경로 표시  | drawMarker(), drawDangerZone(), drawRoute()    |
| ReportForm   | 위험지역 신고 입력 폼 관리  | collectInput(), previewPhoto(), submitReport() |
| AdminView    | 관리자 신고 검토 화면     | loadReports(), approveReport(), rejectReport() |

##### 핵심 메소드 설계

1. submitReport()는 사용자가 신고하기 버튼을 눌렀을 때 실행된다. 위험 유형, 위험도, 위치, 사진, 설명을 수집하고 필수값을 확인한 뒤 ReportService 로 신고 요청을 보낸다.
2. drawDangerZone()은 서버에서 받은 위험 구역 정보를 지도 위에 표시한다. 위험도에 따라 마커 또는 영역의 표시 방식을 다르게 적용하고, 사용자가 선택하면 상세 팝업을 연결한다.

#### 3.2 SDD-P-002 : User/Auth 패키지

User/Auth 패키지는 사용자 계정과 인증, 권한 검증을 담당한다. SafeRoute 의 사용자는 피난민, 군인, 구조기관, 관리자로 구분되므로 사용자 유형에 따라 접근 가능한 화면과 기능이 달라져야 한다. 따라서 인증 기능과 권한 정책을 분리하여 구현한다.

##### 주요 책임

- 회원가입과 로그인 처리
- 비밀번호 해시 처리 및 인증 토큰 발급
- 사용자 유형별 메뉴 접근 제어

- 비활성 또는 제한 계정 접근 차단

### 구성 클래스

| 클래스          | 설명                 | 주요 메소드                                      |
|--------------|--------------------|---------------------------------------------|
| User         | 사용자 계정과 유형을 표현     | changeStatus(), isActive(), hasRole()       |
| AuthService  | 회원가입, 로그인, 로그아웃 처리 | register(), login(), logout(), issueToken() |
| AccessPolicy | 사용자 유형별 권한 판단      | canAccess(), requireAdmin(), requireLogin() |

### 핵심 메소드 설계

1. register()는 회원가입 요청에서 아이디 중복 여부와 필수 입력값을 확인한 뒤 사용자 정보를 저장한다.
2. login()은 계정 정보와 비밀번호를 검증하고, 계정 상태가 정상인 경우 인증 토큰 또는 세션을 생성한다.
3. canAccess()는 사용자의 유형과 요청 기능을 비교하여 접근 가능 여부를 반환한다.

## 3.3 SDD-P-003 : Risk Report 패키지

Risk Report 패키지는 SafeRoute의 핵심 기능인 위험지역 신고를 담당한다. 이 패키지는 개별 신고 데이터를 저장하는 것뿐 아니라, 인접한 위치의 유사 신고를 하나의 위험 구역으로 묶어 지도에 표시하기 쉬운 형태로 변환한다. 이를 통해 많은 사용자가 같은 지역을 신고하더라도 지도 화면이 복잡해지지 않도록 한다.

### 주요 책임

- 위험지역 신고 등록
- 사진과 설명 데이터 처리
- 동일 위치 유사 신고 병합
- 위험 구역의 위험도와 상태 갱신

### 구성 클래스

| 클래스               | 설명              | 주요 메소드                                                |
|-------------------|-----------------|-------------------------------------------------------|
| RiskReport        | 개별 위험지역 신고 정보   | submit(), validate(), markDuplicate(), updateStatus() |
| ReportService     | 신고 등록과 상세 조회 처리 | createReport(), getReportDetail(), listReports()      |
| DangerZoneService | 위험 구역 생성과 병합 처리 | mergeReports(), updateSeverity(), findNearbyZones()   |

### 핵심 메소드 설계

1. createReport()는 신고 입력값을 검증하고 사진 저장소에 첨부 파일을 저장한 뒤 신고 데이터를 DB에 기록한다.
2. mergeReports()는 새 신고와 기존 위험 구역의 거리, 위험 유형, 신고 시간을 비교하여 병합 여부를 판단한다.
3. findNearbyZones()는 현재 위치 반경 내 활성 위험 구역 목록을 조회하여 지도 표시와 경로 탐색에 제공한다.



### 3.4 SDD-P-004 : Map/Route 패키지

Map/Route 패키지는 사용자의 현재 위치 확인, 위험지역 지도 표시, 안전 경로 탐색, 이동 중 재탐색을 담당한다. 이 패키지는 외부 지도 API 와 직접 연결되므로 외부 API 장애에 대한 예외 처리와 캐시 전략이 중요하다.

#### 주요 책임

- GPS 기반 현재 위치 확인
- 위험지역 지도 시각화
- 출발지와 목적지 기반 후보 경로 조회
- 위험 점수 계산을 통한 안전 경로 선택
- 새 위험지역 발생 시 재탐색 제안

#### 구성 클래스

| 클래스             | 설명                 | 주요 메소드                                                              |
|-----------------|--------------------|---------------------------------------------------------------------|
| LocationService | 현재 위치 확인과 수동 위치 처리 | getCurrentLocation(),<br>validateAccuracy(),<br>useManualLocation() |
| MapService      | 지도 표시 데이터 생성       | getMapData(), getMarkerList(),<br>getFilteredZones()                |
| RouteService    | 안전 경로 탐색과 재탐색 처리   | searchRoute(), reroute(),<br>compareRouteWithRisk()                 |
| SafeRouteEngine | 후보 경로의 위험 점수 계산    | scoreRoute(), calculateSafeRoute(),<br>chooseBestRoute()            |

#### 핵심 메소드 설계

1. getCurrentLocation()은 위치 권한을 확인하고 GPS 좌표와 정확도 값을 받아 현재 위치를 지도에 표시한다.
2. searchRoute()는 외부 지도 API 에서 후보 경로를 받은 뒤 위험지역 데이터와 비교하여 안전 경로를 계산한다.
3. reroute()는 이동 중 새 위험지역이 현재 경로와 겹치는 경우 대체 경로를 다시 계산한다.

### 3.5 SDD-P-005 : Shelter/Notification 패키지

Shelter/Notification 패키지는 대피소 정보 제공과 위험 알림을 담당한다. 사용자가 위험지역을 피하는 것뿐 아니라 실제로 대피할 수 있는 장소를 확인할 수 있도록 주변 대피소와 병원, 안전 지역 정보를 제공한다. 또한 경로 중 위험 접근이 감지되면 알림을 통해 사용자에게 즉시 알려준다.

#### 주요 책임

- 현재 위치 주변 대피소 조회
- 대피소 상세 정보와 경로 연결
- 위험 접근 알림 생성
- 경로 재탐색 제안 알림
- 신고 상태 변경 알림

#### 구성 클래스

| 클래스 | 설명 | 주요 메소드 |
|-----|----|--------|
|-----|----|--------|

| 클래스                 | 설명               | 주요 메소드                                                      |
|---------------------|------------------|-------------------------------------------------------------|
| Shelter             | 대피소 또는 안전 지역 정보  | isAvailable(), distanceFrom(),<br>updateStatus()            |
| ShelterService      | 대피소 목록과 상세 정보 제공 | findNearbyShelters(),<br>getShelterDetail(), connectRoute() |
| NotificationService | 위험 및 상태 알림 생성    | sendRiskAlert(), sendRerouteAlert(),<br>markRead()          |

### 핵심 메소드 설계

1. findNearbyShelters()는 현재 위치를 기준으로 반경 내 대피소를 조회하고 운영 상태와 거리를 기준으로 정렬한다.
2. connectRoute()는 사용자가 특정 대피소를 선택하면 현재 위치에서 해당 대피소까지의 경로 탐색을 RouteService 에 요청한다.
3. sendRiskAlert()는 사용자가 위험지역에 접근하거나 새로운 위험지역이 경로와 겹칠 때 경고 알림을 생성한다.

## 3.6 SDD-P-006 : Admin/Data 패키지

Admin/Data 패키지는 관리자 기능과 데이터 관리 기능을 담당한다. SafeRoute 는 사용자 신고 기반 시스템이므로 허위 신고나 중복 신고를 관리할 수 있어야 한다. 관리자는 신고 상세 정보를 확인하고 승인, 보류, 삭제 처리를 수행하며, 필요 시 부적절한 사용자를 제한한다.

### 주요 책임

- 신고 목록 검토와 상태 변경
- 사용자 계정 제한
- 관리자 조치 이력 기록
- 정기 백업 및 복구
- 오류 로그와 시스템 변경 이력 관리

### 구성 클래스

| 클래스            | 설명               | 주요 메소드                                                          |
|----------------|------------------|-----------------------------------------------------------------|
| AdminService   | 신고 검토와 사용자 관리    | reviewReport(), approveReport(),<br>rejectReport(), blockUser() |
| RiskRepository | 신고와 위험구역 DB 접근   | saveReport(), findReports(),<br>updateStatus(), findZones()     |
| BackupService  | 데이터 백업과 복구 관리    | backup(), restore(), verifyBackup()                             |
| AuditLogger    | 관리자 조치와 오류 로그 기록 | writeLog(), findLogs(), exportLogs()                            |

### 핵심 메소드 설계

1. reviewReport()는 관리자가 신고 상세 정보를 확인하고 승인, 보류, 삭제 중 하나의 상태를 선택하면 신고 상태를 변경하고 로그를 기록한다.
2. backup()은 정해진 시간에 DB 스냅샷을 생성하고 외부 저장소에 저장한 뒤 백업 결과를 로그에 남긴다.
3. writeLog()는 데이터 변경, 관리자 조치, 오류 발생 시 변경 전후 값을 기록하여 이후 감사와 장애 분석에 활용한다.

## 4. 인터페이스 설계

인터페이스 설계는 SafeRoute 가 외부 시스템과 데이터를 주고받는 방식과 사용자가 화면을 통해 시스템을 조작하는 방식을 정의한다. 본 시스템은 지도 API, GPS 위치 서비스, 공공 대피소 데이터, 이미지 저장소, 푸시 알림 서비스와 연동되므로 각 연동 지점의 책임과 데이터 형식을 명확히 해야 한다.

### 4.1 외부 시스템 인터페이스

외부 시스템 인터페이스는 서버 내부의 Service 가 직접 호출하지 않고 Adapter 를 통해 접근하도록 설계한다. 이렇게 하면 지도 API 나 알림 서비스가 변경되더라도 핵심 비즈니스 로직의 수정 범위를 줄일 수 있다. 모든 외부 통신은 HTTPS 를 기본으로 하며, API Key 와 인증 정보는 코드에 직접 작성하지 않고 환경 변수 또는 설정 파일로 관리한다.

| 외부 시스템     | 연동 모듈               | 송수신 데이터                    | 방식             | 예외 처리                        |
|------------|---------------------|----------------------------|----------------|------------------------------|
| 지도 API     | Map/Route           | 지도 타일, 위치 검색, 후보 경로, 좌표 변환 | REST/SDK       | 호출 실패 시 오류 안내 및 최근 지도 데이터 사용 |
| GPS/위치 서비스 | LocationService     | 현재 위도, 경도, 정확도, 위치 권한 상태   | Device API     | 권한 거부 시 수동 위치 입력 제공          |
| 공공 대피소 데이터 | ShelterService      | 대피소명, 좌표, 수용 가능 인원, 운영 상태  | REST/Batch     | 데이터 지연 시 최근 캐시 데이터 사용        |
| 클라우드 DB    | Repository          | 사용자, 신고, 위험구역, 경로 이력, 로그   | SQL/ORM        | DB 장애 시 오류 안내 및 복구 절차 수행     |
| 이미지 저장소    | ReportService       | 신고 사진 파일, 파일 URL, 메타데이터    | Object Storage | 파일 형식과 용량 제한, 악성 파일 검사       |
| 푸시 알림 서비스  | NotificationService | 위험 접근 알림, 재탐색 알림, 신고 상태 알림 | Push API       | 발송 실패 시 재시도 또는 알림 이력 보관      |

### 4.2 사용자 인터페이스

사용자 인터페이스는 전쟁 또는 재난 상황에서도 빠르게 사용할 수 있도록 단순한 흐름을 우선한다. 메인 지도 화면에서 현재 위치, 위험지역, 신고 버튼, 경로 탐색 버튼, 대피소 버튼을 한눈에 확인할 수 있어야 한다. 지도 위 위험지역은 위험도에 따라 구분되며, 사용자가 마커를 누르면 위험 유형, 신고 시간, 설명, 신뢰도 정보를 팝업으로 확인할 수 있다.

위험지역 신고 화면은 사용자가 오래 고민하지 않고 입력할 수 있도록 위험 유형 선택, 위험도 선택, 위치 확인, 사진 첨부, 설명 입력 순서로 구성한다. 필수 입력값이 누락되면 즉시 안내 메시지를 제공하고, 네트워크가 불안정할 경우 임시 저장 또는 재시도 안내를 제공한다.

안전 경로 탐색 화면은 출발지와 목적지를 입력하면 지도 위에 추천 경로를 표시하고 예상 거리, 예상 시간, 위험 회피 여부를 함께 보여준다. 이동 중 새로운 위험지역이 발생하면 화면 상단 또는 알림 팝업으로 재탐색을 제안한다. 관리자 화면은 신고 목록, 신고 상세, 사용자 관리, 로그 확인 영역으로 나누어 운영자가 빠르게 검토할 수 있도록 설계한다.

| 화면       | 주요 입력                        | 주요 출력                 | 관련 Use Case |
|----------|------------------------------|-----------------------|-------------|
| 로그인/회원가입 | 아이디, 비밀번호, 사용자 유형            | 로그인 결과, 권한별 메인 화면     | U_01        |
| 메인 지도    | 현재 위치 버튼, 필터, 신고, 경로, 대피소 버튼 | 현재 위치, 위험지역 마커, 추천 경로 | U_03, U_04  |
| 위험지역 신고  | 위험 유형, 위험도, 위치, 사진, 설명       | 신고 접수 결과, 지도 반영 결과    | U_02        |
| 안전 경로 탐색 | 출발지, 목적지, 대피소 경유 여부          | 추천 경로, 예상 거리, 예상 시간   | U_05, U_07  |
| 대피소 정보   | 현재 위치 또는 선택 위치               | 대피소 목록, 상세 정보, 경로 연결  | U_06        |

| 화면  | 주요 입력                | 주요 출력                | 관련 Use Case |
|-----|----------------------|----------------------|-------------|
| 관리자 | 신고 필터, 처리 상태, 사용자 조건 | 승인/보류/삭제, 사용자 제한, 로그 | U_08, U_09  |

## 5. 데이터 설계

SafeRoute의 데이터 설계는 위치 기반 데이터의 정확성, 위험 정보의 실시간 갱신, 사용자 개인정보 보호, 신고 검증 이력 보존을 중심으로 구성한다. 데이터베이스는 관계형 DB를 기준으로 설계하되, 위험지역과 대피소처럼 좌표 기반 검색이 많은 데이터는 지리 공간 인덱스를 적용하는 것을 고려한다.

사용자가 입력한 신고는 RiskReport 테이블에 개별 신고 단위로 저장된다. 이후 동일 위치 또는 인접 위치의 유사 신고는 DangerZone으로 병합되어 지도에 표시된다. 이 구조는 신고 원본을 보존하면서도 사용자 화면에는 정리된 위험 구역을 보여줄 수 있다는 장점이 있다.

| 엔티티          | 설명                  | 주요 속성                                                                                                        |
|--------------|---------------------|--------------------------------------------------------------------------------------------------------------|
| User         | 사용자 계정과 사용자 유형 관리   | user_id, login_id, password_hash, user_type, status, created_at                                              |
| RiskReport   | 사용자가 등록한 개별 위험지역 신고 | report_id, reporter_id, risk_type, severity, latitude, longitude, photo_url, description, status, created_at |
| DangerZone   | 지도에 표시되는 위험 구역      | zone_id, center_lat, center_lng, radius, severity, report_count, status, last_updated_at                     |
| RouteHistory | 경로 탐색 요청과 결과 이력     | route_id, user_id, start_lat, start_lng, end_lat, end_lng, distance, duration, risk_score, created_at        |
| Shelter      | 대피소 또는 안전 지역 정보     | shelter_id, name, latitude, longitude, capacity, status, contact, updated_at                                 |
| Notification | 사용자 알림 정보           | notification_id, user_id, type, message, target_id, read_status, created_at                                  |
| AuditLog     | 관리자 조치와 시스템 변경 이력   | log_id, actor_id, action_type, target_table, target_id, before_value, after_value, created_at                |

### 5.1 저장 방식

사용자 정보는 관계형 DB에 저장하며, 비밀번호는 원문으로 저장하지 않고 해시 처리한다. 위치 정보는 서비스 제공에 필요한 범위에서만 사용하고, 사용자 권한과 접근 로그를 통해 무단 조회를 방지한다. 신고 사진은 DB에 직접 저장하지 않고 이미지 저장소에 업로드한 뒤 URL만 DB에 보관한다. 이를 통해 DB 용량 증가를 줄이고 이미지 파일 관리 효율성을 높인다.

위험지역 신고와 위험 구역 데이터는 경로 탐색과 지도 표시에서 자주 사용되므로 검색 성능이 중요하다. 따라서 좌표 기반 검색이 가능한 인덱스를 적용하고, 최근 위험지역 목록은 캐시를 사용할 수 있도록 설계한다. 대피소 데이터는 공공 데이터 연동이 지연될 수 있으므로 주기적으로 동기화하여 최신 데이터를 저장하고, 네트워크 장애 시에는 마지막으로 저장된 데이터를 제한적으로 제공한다.

### 5.2 데이터 무결성 및 보안

데이터 무결성을 위해 신고 등록 시 위험 유형, 위험도, 좌표, 첨부 파일 형식과 용량을 서버에서 검증한다. 신고 상태는 접수, 검토 중, 승인, 보류, 삭제 또는 해제처럼 정해진 값만 사용할 수 있도록 제한한다. 위험도는 지도 표시와 경로 계산에 직접 영향을 주므로 허용된 값만 저장되도록 한다.

보안을 위해 관리자 기능은 관리자 권한을 가진 사용자만 접근할 수 있도록 제한한다. 사용자의 위치 정보와 계정 정보는 최소 권한 원칙에 따라 접근을 제한하고, 관리자가 신고 상태를 변경하거나 사용자를 제한하는 경우 AuditLog에 변경 전후 값을 기록한다. 백업 데이터 역시 외부 유출을 막기 위해 접근 권한을 분리한다.

## 6. 구현 기술 설계

구현 기술 설계는 SafeRoute 를 실제로 개발할 때 필요한 기술, 개발 환경, 적용 전략을 정리한 것이다. SafeRoute 는 지도 기반 서비스이므로 클라이언트 화면과 위치 API 연동이 중요하며, 사용자 신고가 실시간으로 반영되어야 하므로 서버와 데이터베이스 설계도 함께 고려해야 한다.

| 구분     | 기술                                       | 적용 내용                               |
|--------|------------------------------------------|-------------------------------------|
| 클라이언트  | Kotlin Android 또는 반응형 Web UI             | 지도 화면, 신고 화면, 경로 탐색 화면, 알림 화면 구현    |
| 백엔드    | Spring Boot(Java/Kotlin) REST API        | 인증, 신고 처리, 경로 계산, 대피소 조회, 관리자 기능 제공 |
| 데이터베이스 | PostgreSQL/PostGIS 또는 클라우드 DB            | 위치 좌표 기반 검색, 신고 데이터 저장, 위험구역 관리     |
| 지도/경로  | Kakao/Naver/Google Map API 중 선택          | 지도 표시, 위치 검색, 경로 후보 조회, 마커 표시       |
| 파일 저장  | Cloud Object Storage 또는 Firebase Storage | 신고 사진 저장 및 URL 관리                   |
| 알림     | Firebase Cloud Messaging 또는 자체 알림 모듈     | 위험 접근, 경로 재탐색, 신고 상태 변경 알림          |
| 버전 관리  | Git, GitHub                              | 문서와 코드 커밋, 브랜치 관리, 과제 산출물 보관        |
| 테스트    | JUnit, Postman, Android Emulator         | 단위 테스트, API 테스트, 화면 동작 확인           |

### 6.1 개발 환경

개발 환경은 Windows 10/11 을 기본으로 하며, 서버 배포 또는 DB 운영 환경이 필요한 경우 Linux 기반 서버 환경을 사용할 수 있다. IDE 는 IntelliJ IDEA, Android Studio, Visual Studio Code 를 활용하고, 빌드 도구는 Gradle 또는 Maven 을 프로젝트 구조에 맞게 선택한다. API 테스트는 Postman 또는 Swagger UI 를 사용하여 요청과 응답을 검증한다.

### 6.2 적용 기술 및 설계 전략

클라이언트와 서버는 RESTful API 방식으로 통신한다. 각 기능은 URL 과 HTTP Method 를 기준으로 구분하고, 요청과 응답은 JSON 형식을 기본으로 한다. 로그인 이후에는 JWT 또는 세션 인증을 적용하여 사용자 유형과 권한을 확인한다. 위치 기반 검색은 현재 위치 반경 내 위험지역과 대피소를 빠르게 조회할 수 있도록 좌표 인덱스를 적용한다.

안전 경로 탐색은 외부 지도 API 가 제공하는 후보 경로를 그대로 사용하는 것이 아니라, SafeRouteEngine 에서 위험지역과의 거리, 위험도, 최근 신고 여부, 대피소 경유 가능성을 종합하여 위험 점수를 계산하는 방식으로 설계한다. 이를 통해 단순히 최단 경로가 아니라 위험지역을 최대한 피하는 경로를 추천할 수 있다.

### 6.3 예외 처리 및 장애 대응

네트워크 장애가 발생하면 사용자가 작성한 신고 데이터를 임시 저장하거나 재시도 안내를 제공한다. GPS 권한이 거부된 경우에는 수동 위치 입력 기능을 제공하고, 지도 API 호출이 실패하면 오류 메시지를 표시한 뒤 최근 캐시된 위험지역 정보를 제한적으로 제공한다. DB 장애가 발생하면 쓰기 기능을 제한하고 복구 후 동기화하는 방식을 고려한다.

허위 신고는 사용자 신고 기반 시스템에서 중요한 위험 요소이다. 따라서 중복 신고 분석, 관리자 검토, 신고 상태 보류 및 삭제 기능을 통해 신뢰성을 관리한다. 관리자 조치 내역은 AuditLog 에 저장하여 이후 검토와 문제 해결에 활용한다.



## 7. 요구사항 추적표

요구사항 추적표는 요구사항 정의서와 요구사항 분석서에서 도출된 요구사항이 설계 요소와 어떻게 연결되는지를 나타낸다. 이 표를 통해 구현 및 테스트 단계에서 누락된 기능을 확인할 수 있으며, 각 요구사항이 어떤 패키지와 클래스, 화면, 데이터 요소로 구현되는지 추적할 수 있다.

| 요구사항 ID | 요구사항명            | 설계 패키지                          | 설계 요소                                         | UI/화면       |
|---------|------------------|---------------------------------|-----------------------------------------------|-------------|
| FR-001  | 회원가입             | User/Auth                       | AuthService.register(), User                  | 로그인/회원가입 화면 |
| FR-002  | 로그인              | User/Auth                       | AuthService.login(), AccessPolicy             | 로그인 화면      |
| FR-003  | 사용자 유형 구분        | User/Auth, Client/UI            | AccessPolicy.canAccess(), User.user_type      | 권한별 메뉴      |
| FR-004  | 신고 내역 확인         | Risk Report                     | ReportService.listReports()                   | 마이페이지/신고 목록 |
| FR-005  | 사용자 계정 제한        | Admin/Data                      | AdminService.blockUser(), AuditLog            | 관리자 사용자 화면  |
| FR-006  | 현재 위치 기반 위험지역 신고 | Risk Report, Map/Route          | ReportService.createReport(), LocationService | 위험지역 신고 화면  |
| FR-007  | 지도 선택 위치 신고      | Client/UI, Risk Report          | MapView.selectLocation(), createReport()      | 지도 선택 UI    |
| FR-008  | 위험 유형 선택         | Risk Report                     | RiskReport.risk_type                          | 신고 입력 폼     |
| FR-009  | 위험도 설정           | Risk Report, Map/Route          | RiskReport.severity, MapView.drawDangerZone() | 지도 마커/영역    |
| FR-010  | 사진 및 설명 첨부       | Risk Report                     | ReportService.createReport(), FileStorage     | 사진 첨부 UI    |
| FR-011  | 신고 접수 시간 자동 기록   | Risk Report                     | RiskRepository.saveReport()                   | 신고 완료 화면    |
| FR-012  | 중복 신고 병합         | Risk Report                     | DangerZoneService.mergeReports()              | 지도 위험구역     |
| FR-013  | GPS 현재 위치 확인     | Map/Route                       | LocationService.getCurrentLocation()          | 현재 위치 버튼    |
| FR-014  | 주변 위험지역 자동 조회    | Map/Route, Risk Report          | DangerZoneService.findNearbyZones()           | 메인 지도       |
| FR-015  | 가까운 안전 지역 확인     | Shelter/Notification            | ShelterService.findNearbyShelters()           | 대피소 화면      |
| FR-016  | 수동 위치 입력         | Map/Route, Client/UI            | LocationService.useManualLocation()           | 수동 위치 입력 UI |
| FR-017  | 위험지역 지도 표시       | Map/Route                       | MapView.drawDangerZone(), MapService          | 메인 지도       |
| FR-018  | 위험도별 색상 구분       | Client/UI, Map/Route            | MapView.drawMarker()                          | 지도 마커       |
| FR-019  | 위험 유형 필터         | Map/Route                       | MapService.getFilteredZones()                 | 필터 패널       |
| FR-020  | 최신 정보 확인         | Risk Report                     | ReportService.listReports()                   | 최근 신고 정렬    |
| FR-021  | 위험지역 상세 확인       | Client/UI, Risk Report          | ReportService.getReportDetail()               | 위험지역 팝업     |
| FR-022  | 출발지/목적지 입력       | Map/Route                       | RouteRequest.validate()                       | 경로 탐색 화면    |
| FR-023  | 위험지역 회피 경로 계산    | Map/Route                       | SafeRouteEngine.calculateSafeRoute()          | 추천 경로 화면    |
| FR-024  | 예상 거리/시간 제공      | Map/Route                       | RouteService.searchRoute()                    | 경로 상세 패널    |
| FR-025  | 새 위험 발생 시 재탐색 제안 | Map/Route, Notification         | RouteService.reroute(), NotificationService   | 알림/재탐색 팝업   |
| FR-026  | 추천 경로 지도 표시      | Client/UI                       | MapView.drawRoute()                           | 지도 경로 선     |
| FR-027  | 대피소 목록 제공        | Shelter/Notification            | ShelterService.findNearbyShelters()           | 대피소 목록      |
| FR-028  | 대피소 상세 정보        | Shelter/Notification            | ShelterService.getShelterDetail()             | 대피소 상세 화면   |
| FR-029  | 대피소 경로 연결        | Shelter/Notification, Map/Route | ShelterService.connectRoute()                 | 경로 연결 버튼    |
| FR-030  | 신고 검증 및 상태 관리    | Admin/Data                      | AdminService.reviewReport(), AuditLog         | 관리자 신고 화면   |
| NFR-001 | 실시간성             | 전체 패키지                          | API 응답 최적화, 캐시, 알림                            | 지도/알림 UI    |
| NFR-002 | 보안성              | User/Auth, Admin/Data           | AccessPolicy, passwordHash, AuditLogger       | 권한 분리 UI    |
| NFR-003 | 사용성              | Client/UI                       | UIController, MapView                         | 전체 사용자 화면   |
| IR-001  | 지도 API 연동        | Map/Route                       | MapApiAdapter                                 | 지도 화면       |
| IR-002  | GPS 위치 서비스 연동    | Map/Route                       | GPSAdapter, LocationService                   | 현재 위치 표시    |



## 8. 부록

### 8.1 GitHub Repository 관리 계획

프로젝트 관련 문서와 코드, 추가 자료는 GitHub Repository에 함께 관리한다. 문서 파일은 버전과 작성일을 포함한 파일명으로 저장하고, 기능별 구현 파일은 패키지 구조에 맞추어 분리한다. 과제 제출 시에는 문서 산출물뿐 아니라 README, API 명세, DB 스크립트, 테스트 자료 등을 함께 관리하면 프로젝트의 완성도를 높일 수 있다.

| 폴더        | 관리 내용                                               |
|-----------|-----------------------------------------------------|
| /docs     | 시스템 계획서, 요구사항 정의서, 요구사항 분석서, 소프트웨어 설계서 등 문서 산출물 보관  |
| /src      | 클라이언트와 서버 소스 코드 보관                                  |
| /api      | API 명세서, 요청/응답 예시, Swagger 또는 Postman Collection 보관 |
| /database | ERD, 테이블 생성 스크립트, 샘플 데이터 보관                         |
| /assets   | UI 이미지, 아이콘, 발표 자료, 설계 다이어그램 보관                     |
| /test     | 테스트 케이스, 테스트 결과, 오류 수정 내역 보관                        |

### 8.2 향후 확장사항

향후에는 네트워크가 불안정한 상황을 고려하여 오프라인 모드를 추가할 수 있다. 오프라인 모드에서는 최근 위험지역 지도와 대피소 데이터를 캐시하여 제한적으로 제공하고, 사용자가 작성한 신고는 네트워크가 복구된 뒤 서버에 전송한다.

또한 신고 신뢰도 점수 기능을 확장할 수 있다. 신고자의 과거 신고 이력, 동일 위치 반복 신고 여부, 관리자 검토 결과를 종합하여 신뢰도를 계산하면 허위 신고를 줄이고 실제 위험 정보의 활용도를 높일 수 있다. 장기적으로는 구조기관 전용 대시보드, 다국어 지원, AI 기반 위험 예측 기능도 추가 가능하다.

### 8.3 검토 체크리스트

| 항목     | 점검 내용                            | 결과 |
|--------|----------------------------------|----|
| 서론     | 목적 및 범위, 용어 정의, 참조 문서가 포함되었는가?   | 확인 |
| 아키텍처   | 정적 구조와 동적 구조가 모두 설명되었는가?         | 확인 |
| 모듈 설계  | 각 패키지별 상세 설계 내용이 포함되었는가?         | 확인 |
| 인터페이스  | 외부 시스템 인터페이스와 사용자 인터페이스가 포함되었는가? | 확인 |
| 데이터 설계 | 데이터 구조와 저장 방식이 포함되었는가?           | 확인 |
| 구현 기술  | 사용 기술, 개발 환경, 적용 기술이 포함되었는가?     | 확인 |
| 추적표    | 요구사항과 설계 요소 간 추적 관계가 포함되었는가?     | 확인 |